

SLOT7:

EJS, EJS with node and express, Templating with EJS I

Tóm tắt nội dung

- Embedded Javascript (EJS):
- 19 What is EJS?
- 20 Understand what EJS does and how to use it with node and express
- 21 Templating with EJS

1. What is EJS?

- **EJS** (Embedded JavaScript) là một **template engine** cho Node.js.
- Nó cho phép **nhúng JavaScript vào HTML** để tạo ra nội dung động.
- Tập EJS có phần mở rộng là `.ejs`.

Ví dụ:

`ejs`

```
<h1>Chào <%= tenNguoiDung %>!/</h1>
```

2. Understand what EJS does and how to use it with Node & Express

EJS làm gì?

- **Chuyển đổi các file `.ejs` thành HTML** với dữ liệu động.
- Cho phép viết JavaScript trong HTML để hiển thị dữ liệu từ backend.

Cách sử dụng với Node & Express:

1. Cài đặt EJS:

```
bash
```

```
npm install ejs
```

2. Thiết lập EJS trong Express:

```
js
```

```
const express = require('express');  
const app = express();
```

```
app.set('view engine', 'ejs');
```

3. Tạo file `.ejs` trong thư mục `views`:

Ví dụ: `views/trangchu.ejs`

4. Render EJS trong route:

```
js

app.get('/', (req, res) => {
  res.render('trangchu', { tenNguoiDung: 'Minh' });
});
```

3. Templating with EJS

Các cú pháp phổ biến:

- **Hiển thị giá trị:**

```
ejs

<%= bien %>
```

- **Viết JavaScript logic:**

```
ejs

<% if (dieuKien) { %>
  <p>Đúng rồi!</p>
<% } %>
```

- **Vòng lặp:**

```
ejs

<ul>
  <% danhSach.forEach(item => { %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

Ưu điểm:

- Dễ học, dễ sử dụng cho người mới.
- Làm việc tốt với Express.
- Không cần phải học framework phức tạp (như React) khi mới bắt đầu.

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

1. ✓ Hiểu khái niệm EJS là gì

- EJS là **template engine** cho phép nhúng **JavaScript** vào **HTML**.
- Dùng để tạo **giao diện động** (dynamic HTML) từ dữ liệu backend.

2. ✓ Cách tích hợp EJS với Express

- Cài đặt bằng `npm install ejs`
- Thiết lập EJS trong ứng dụng Express bằng:

```
js
app.set('view engine', 'ejs');
```

- Tạo thư mục `views/` để chứa các file `.ejs`.

3. ✓ Cách truyền và hiển thị dữ liệu từ server vào EJS

- Truyền dữ liệu qua `res.render()`:

```
js
res.render('ten_file', { bien1: 'giatri1', bien2: [1,2,3] });
```

- Hiển thị dữ liệu trong EJS bằng cú pháp:

```
ejs
<%= bien1 %>    <!-- In ra giá trị -->
<% if (dieuKien) { %>    <!-- Logic điều kiện -->
...
<% } %>
```

4. ✓ Cú pháp EJS cơ bản cần nắm

Cú pháp	Mục đích
<code><%= ... %></code>	Hiển thị dữ liệu (escape HTML)
<code><%- ... %></code>	Hiển thị dữ liệu không escape HTML
<code><% ... %></code>	Chạy logic JavaScript (không in)

5. ✓ Tách giao diện thành các thành phần nhỏ (partials)

- Dùng để **tái sử dụng HTML** (header, footer, navbar...).
- Gọi partials bằng cú pháp:

```
ejs
<%- include('partials/header') %>
```

6. Không nhầm lẫn giữa EJS và các frontend framework như React

- EJS chạy **trên server** (server-side rendering).
- Không chạy được trong trình duyệt nếu không qua Express.

7. Thư mục `views` là mặc định cho EJS

- Express sẽ tìm các file `.ejs` trong thư mục `views/` theo mặc định.

ỨNG DỤNG THỰC TIỄN:

1. Tạo giao diện người dùng động (Dynamic UI)

- Dựa vào dữ liệu từ server (ví dụ từ cơ sở dữ liệu), bạn có thể **hiển thị nội dung khác nhau tùy người dùng**.
- Ví dụ:
 - Hiển thị tên người dùng sau khi đăng nhập.
 - Hiển thị danh sách sản phẩm từ database.

`ejs`

```
<h2>Xin chào <%= user.name %></h2>
<ul>
  <% products.forEach(product => { %>
    <li><%= product.title %> - $<%= product.price %></li>
  <% }) %>
</ul>
```

2. Tạo các trang HTML có cấu trúc lặp lại (reuse với partials)

- Tái sử dụng header, footer, navbar, v.v.
- Giúp dự án dễ quản lý, tiết kiệm thời gian viết lại HTML lặp đi lặp lại.

`ejs`

```
<%- include('partials/header') %>
<main>Trang nội dung</main>
<%- include('partials/footer') %>
```

3. Dựng nhanh các website thương mại, blog, hệ thống quản lý...

- EJS phù hợp với các dự án cần:
 - Tạo giao diện nhanh.
 - Không cần SPA (single-page app) quá phức tạp như React.
 - Xử lý server-side rendering.

Ví dụ:

- Website bán hàng nhỏ.
- Blog cá nhân.
- Hệ thống quản lý (CMS) đơn giản.

4. 🛡️ Tương tác với người dùng

- Hiện thị lỗi đăng nhập, thông báo thành công, form nhập liệu,...

ejs

```
<% if (error) { %>
  <p class="error"><%= error %></p>
<% } %>
```

5. 🌐 Tích hợp dễ dàng với các hệ thống backend

- EJS hoạt động mượt mà với Express.
- Dễ tích hợp với cơ sở dữ liệu (MongoDB, MySQL,...).
- Phù hợp cho MVP (Minimum Viable Product) – dự án thử nghiệm nhanh.

✅ Tóm lại

EJS rất thích hợp để **xây dựng các ứng dụng web có giao diện động và đơn giản** mà vẫn đủ mạnh mẽ để làm việc với dữ liệu thật, từ backend đến frontend.

CHI TIẾT MỘT SỐ NỘI DUNG:

1. Tổng quan & Bản chất của EJS

EJS là gì?

- **EJS (Embedded JavaScript)** là một **template engine** dùng để tạo **HTML động**.
- Cho phép bạn viết **JavaScript ngay bên trong mã HTML**, giúp nội dung hiển thị tùy thuộc vào dữ liệu từ server.

Tại sao cần dùng EJS?

- Khi người dùng truy cập website, server sẽ lấy dữ liệu và gửi về **HTML đã được “render” sẵn**.
- Thay vì viết HTML bằng tay cho từng trang, EJS giúp bạn **tự động hóa quá trình** đó dựa trên dữ liệu.

2. Cách hoạt động của EJS trong Express

Các bước tích hợp EJS:

1. Cài đặt EJS:

```
bash
```

```
npm install ejs
```

2. Thiết lập Express để dùng EJS:

```
js
```

```
const express = require('express');  
const app = express();  
app.set('view engine', 'ejs'); // Thiết lập EJS là engine mặc định
```

3. Cấu trúc thư mục:

```
pgsql
```

```
project/  
├── views/  
│   ├── index.ejs  
│   └── partials/  
│       ├── header.ejs  
│       └── footer.ejs  
├── public/           # Static files (CSS, JS, images)  
└── app.js             # Entry point
```

4. Render file EJS:

```
js
```

```
app.get('/', (req, res) => {
```

```
res.render('index', { title: 'Trang chủ', user: { name: 'Minh' } });
});
```

3. Cú pháp EJS – Phân tích chi tiết

Cú pháp	Mục đích	Ví dụ
<code><%= value %></code>	In giá trị ra HTML, tự động escape HTML	<code><%= user.name %></code>
<code><%- value %></code>	In ra không escape (cho HTML hoặc raw HTML)	<code><%- htmlSnippet %></code>
<code><% code %></code>	Viết code JS không xuất ra HTML (if, for...)	<code><% if (true) { %></code>
<code>include()</code>	Nhúng file EJS con (partial)	<code><%- include('partials/header') %></code>

Cú pháp logic:

ejs

```
<% if (isLoggedIn) { %>
  <h2>Welcome, <%= user.name %></h2>
<% } else { %>
  <a href="/login">Login</a>
<% } %>
```

4. Tổ chức mã nguồn hiệu quả với Partials & Layouts

Partials:

- Tái sử dụng HTML: header, footer, navbar, v.v.
- Tạo thư mục `views/partials/`, dùng `include()`.

Layouts (EJS không hỗ trợ trực tiếp layout như `handlebars`, nhưng có thể mô phỏng):

ejs

```
<!-- views/layout.ejs -->
<%- include('partials/header') %>
<main>
  <%- body %>
</main>
<%- include('partials/footer') %>
```

Ở file render:

js

```
res.render('trangchu', { body:
ejs.render(fs.readFileSync('views/trangchu.ejs', 'utf8'), data) });
```

(Hoặc dùng package như `ejs-mate` để hỗ trợ layout.)

5. Ứng dụng thực tế & Kết hợp với dữ liệu

Kết nối database (MongoDB chẳng hạn)

```
js

const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/mydb');

const Product = mongoose.model('Product', { name: String, price: Number });

app.get('/products', async (req, res) => {
  const products = await Product.find();
  res.render('products/index', { products });
});
ejs

<!-- views/products/index.ejs -->
<h1>Danh sách sản phẩm</h1>
<ul>
  <% products.forEach(p => { %>
    <li><%= p.name %> - $<%= p.price %></li>
  <% }) %>
</ul>
```

6. Những lỗi phổ biến khi học EJS

Lỗi	Nguyên nhân	Cách khắc phục
res.render() not working	Chưa set view engine	Thêm app.set('view engine', 'ejs')
Không render đúng dữ liệu	Thiếu {} hoặc đặt sai tên biến	Kiểm tra kỹ tên biến, truyền đúng
EJS không nhận file	File .ejs không nằm trong views/	Đảm bảo file đúng thư mục

7. Khi nào nên và không nên dùng EJS

Nên dùng EJS khi:

- Làm web server-side đơn giản.
- Web có ít logic frontend (chủ yếu render HTML từ backend).
- Dự án MVP, demo hoặc nội bộ.
- Bạn đang học web server-side và muốn nắm kiến thức nền tảng.

Không nên dùng khi:

- Làm web phức tạp, cần SPA (ứng dụng 1 trang): dùng React, Vue, Angular.
- Giao diện tương tác mạnh mẽ phía client.

✓ TÓM TẮT CÁC BƯỚC DÙNG EJS TRONG DỰ ÁN NODEJS

◆ BƯỚC 1: Cài đặt EJS

```
bash

npm install ejs
```

◆ BƯỚC 2: Cấu hình EJS trong Express

```
js

const express = require('express');
const app = express();

// Cấu hình template engine là EJS
app.set('view engine', 'ejs');

// (Tuỳ chọn) Chỉ định đường dẫn đến thư mục views
app.set('views', './views');
```

◆ BƯỚC 3: Tạo thư mục `views/` và file `.ejs`

- Ví dụ: `views/index.ejs`

```
ejs

<h1>Chào mừng <%= username %>!</h1>
```

◆ BƯỚC 4: Dùng `res.render()` để render file EJS

```
js

app.get('/', (req, res) => {
  res.render('index', { username: 'Minh' });
});
```

◆ BƯỚC 5: (Tuỳ chọn) Dùng `partials` để tái sử dụng giao diện

- Tạo `views/partials/header.ejs`, `footer.ejs`
- Nhúng bằng:

```
ejs

<%- include('partials/header') %>
<!-- Nội dung -->
<%- include('partials/footer') %>
```

◆ BƯỚC 6: (Tuỳ chọn) Kết hợp với dữ liệu từ database

js

```
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.render('users/index', { users });
});
```

Ghi nhớ nhanh

- `app.set('view engine', 'ejs')` → Bắt buộc
- `res.render('file', data)` → Render file `.ejs` trong thư mục `views/`
- `<%= %>` → In dữ liệu
- `<%- include('file') %>` → Nhúng partial

Tổng kết kiến thức

Mục	Ghi nhớ
Template Engine	EJS là engine giúp render HTML động
Express Integration	Dùng <code>app.set('view engine', 'ejs')</code>
Syntax	<code><%= %></code> , <code><% %></code> , <code><%- %></code> , <code>include()</code>
Tổ chức code	Partials, phân chia logic
Ứng dụng thực tế	Tạo trang chủ, danh sách, blog, login, v.v.
Phù hợp với	Web đơn giản, server-rendered web

KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

1. KINH NGHIỆM THỰC TIỄN

a. Tổ chức thư mục rõ ràng, chuẩn MVC

- Tách biệt phần **routes**, **controllers**, **views**, và **public (static assets)**:

```
bash

/routes
/controllers
/views
├── partials/
└── pages/
/public
app.js
```

- Giúp dễ bảo trì khi dự án lớn hơn.

b. Tái sử dụng component với **partials**

- Các thành phần lặp lại như **header**, **footer**, **navbar**, **form** → đưa vào `views/partials/`.
- Dùng `include()` trong mọi trang:

```
ejs

<%- include('partials/header') %>
```

c. Tách file CSS/JS và dùng static middleware

```
js

app.use(express.static('public'));
```

- Tạo thư mục `public/` chứa `style.css`, `script.js`, hình ảnh, v.v.

2. VẤN ĐỀ CẦN LƯU Ý

a. Escape HTML mặc định – an toàn nhưng cần hiểu rõ

- `<%= %>` tự động escape HTML → tránh XSS.
- Nếu muốn render HTML (ví dụ đoạn `<div>` được lưu trong DB), dùng `<%- %>`.

b. Không nhầm lẫn `res.send()` và `res.render()`

- `res.send()` gửi dữ liệu dạng text hoặc JSON.
- `res.render()` kết hợp view + data và trả về HTML.

c. Sai tên biến khi render → lỗi hoặc trang trắng

- Luôn **kiểm tra dữ liệu đầu vào**.
- Dùng `console.log()` để debug trong controller.

d. Không gọi `res.render()` nhiều lần trong cùng route

```
js
// Sai
if (error) res.render('error');
res.render('success'); // Lỗi vì đã render trước đó
```

- Luôn đảm bảo chỉ render một lần trong mỗi xử lý request.

3. MẸO (TIPS) GIÚP BẠN LÀM VIỆC HIỆU QUẢ HƠN

a. Tạo biến toàn cục dùng nhiều nơi

- Ví dụ: tên app, thông tin đăng nhập, flash message,...

```
js
app.use((req, res, next) => {
  res.locals.appName = 'MyApp';
  res.locals.currentUser = req.user || null;
  next();
});
```

- Dùng trong mọi view:

```
ejs
<title><%= appName %></title>
```

b. Highlight nội dung động hoặc lỗi

```
ejs
<% if (error) { %>
  <p class="error"><%= error %></p>
<% } %>
```

- Dùng biến truyền vào để thông báo lỗi/thành công trong form xử lý người dùng.

c. Viết helper function trong controller

- Nếu có logic xử lý dữ liệu (cắt chuỗi, định dạng giá tiền...), nên xử lý trong controller rồi truyền vào view, không nên viết logic nhiều trong EJS.

💡 d. Kết hợp với layout engine (nếu cần layout nâng cao)

- Dùng thêm thư viện như [ejs-mate](#) để hỗ trợ layout tương tự như trong Handlebars.

bash

```
npm install ejs-mate
```

✓ 4. BONUS – Cách học và thực hành hiệu quả với EJS

Cách học	Lợi ích
Xây 1 mini project (blog, todo, sản phẩm...)	Vừa học cú pháp vừa hiểu luồng dữ liệu
Debug với <code>console.log()</code> và devtools	Nhanh chóng tìm lỗi khi không hiển thị dữ liệu
Tự tách giao diện thành <code>partials</code>	Rèn kỹ năng tổ chức code
Kết hợp MongoDB để lấy dữ liệu thật	Biết cách kết nối backend ↔ frontend

🔙 Tổng kết

EJS rất phù hợp cho người mới học fullstack web development, vì dễ tiếp cận, dễ triển khai, và học được cách tổ chức logic server-side. Tuy không hiện đại như React hay Vue, nhưng là **nền tảng cực tốt** cho mọi lập trình viên backend.

EXERCISE

DỰ ÁN: SỔ TAY QUẢN LÝ GHI CHÚ (NOTE APP)

1. Tóm tắt dự án và mục tiêu

- **Tên:** Note Manager App
- **Mục tiêu:** Tạo một ứng dụng web cho phép người dùng tạo, xem, chỉnh sửa và xóa ghi chú.
- **Người dùng:** Không cần đăng nhập (phiên bản đơn giản)
- **Hiển thị giao diện động bằng EJS và quản lý dữ liệu bằng MongoDB**

2. Chức năng chính

STT	Chức năng
1	Tạo ghi chú mới
2	Hiển thị danh sách ghi chú
3	Xem chi tiết một ghi chú
4	Sửa nội dung ghi chú
5	Xoá ghi chú

3. Công nghệ sử dụng

Công nghệ	Mục đích
Node.js + Express	Backend + server
EJS	Render giao diện HTML động
MongoDB + Mongoose	Quản lý cơ sở dữ liệu
Bootstrap (CDN)	Giao diện đơn giản, responsive
Nodemon	Tự động reload server khi thay đổi code

4. Cấu trúc thư mục dự án

pgsql

```
note-app/  
├── app.js  
├── package.json  
├── /models/  
│   └── Note.js  
├── /routes/  
│   └── notes.js  
├── /views/  
│   ├── layout.ejs  
│   ├── index.ejs  
│   └── notes/  
│       ├── list.ejs  
│       ├── new.ejs  
│       ├── show.ejs  
│       └── edit.ejs  
├── /partials/  
│   ├── header.ejs  
│   └── footer.ejs  
└── /public/
```

5. Tóm tắt các bước thực hiện

Bước	Nội dung
1	Tạo dự án NodeJS, cài đặt thư viện
2	Tạo kết nối MongoDB + mô hình Note
3	Tạo route và controller
4	Tạo các file view với EJS
5	Hoàn thiện giao diện và form
6	Test chức năng

6. Chi tiết các bước thực hiện

BƯỚC 1: Khởi tạo dự án & cài đặt thư viện

bash

```
npm init -y
npm install express mongoose ejs body-parser method-override
npm install --save-dev nodemon
```

BƯỚC 2: Tạo file `app.js`

js

```
const express = require('express');
const mongoose = require('mongoose');
const methodOverride = require('method-override');
const app = express();

// Kết nối DB
mongoose.connect('mongodb://localhost/note-app');

// Middleware
app.use(express.urlencoded({ extended: true }));
app.use(methodOverride('_method'));
app.use(express.static('public'));
app.set('view engine', 'ejs');

// Routes
const noteRoutes = require('./routes/notes');
app.use('/', noteRoutes);

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

BƯỚC 3: Tạo model `Note.js`

js

```
const mongoose = require('mongoose');

const noteSchema = new mongoose.Schema({
  title: String,
```

```

    content: String,
    createdAt: { type: Date, default: Date.now }
  });

module.exports = mongoose.model('Note', noteSchema);

```

BƯỚC 4: Tạo các route trong `routes/notes.js`

```

js

const express = require('express');
const router = express.Router();
const Note = require('../models/Note');

// Trang chủ → danh sách ghi chú
router.get('/', async (req, res) => {
  const notes = await Note.find().sort({ createdAt: -1 });
  res.render('notes/list', { notes });
});

// Form tạo mới
router.get('/notes/new', (req, res) => {
  res.render('notes/new');
});

// Tạo mới ghi chú
router.post('/notes', async (req, res) => {
  const { title, content } = req.body;
  await Note.create({ title, content });
  res.redirect('/');
});

// Xem ghi chú
router.get('/notes/:id', async (req, res) => {
  const note = await Note.findById(req.params.id);
  res.render('notes/show', { note });
});

// Form sửa
router.get('/notes/:id/edit', async (req, res) => {
  const note = await Note.findById(req.params.id);
  res.render('notes/edit', { note });
});

// Cập nhật ghi chú
router.put('/notes/:id', async (req, res) => {
  const { title, content } = req.body;
  await Note.findByIdAndUpdate(req.params.id, { title, content });
  res.redirect('/');
});

// Xóa ghi chú
router.delete('/notes/:id', async (req, res) => {
  await Note.findByIdAndDelete(req.params.id);
  res.redirect('/');
});

module.exports = router;

```


✓ BƯỚC 5: Tạo giao diện với EJS

layout.ejs (layout cơ bản)

ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>Note App</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <%- include('partials/header') %>
  <main>
    <%- body %>
  </main>
  <%- include('partials/footer') %>
</body>
</html>
```

list.ejs, new.ejs, show.ejs, edit.ejs

Mỗi file hiển thị giao diện danh sách, tạo mới, xem chi tiết, chỉnh sửa

✓ 7.05 TEST CASE

TC#	Test Case	Kỳ vọng
1	Truy cập / và hiển thị danh sách ghi chú	Hiển thị danh sách từ DB
2	Gửi form tạo mới ghi chú	Ghi chú được tạo và chuyển hướng về /
3	Truy cập /notes/:id	Hiển thị nội dung ghi chú cụ thể
4	Sửa ghi chú và lưu	Nội dung được cập nhật
5	Xoá ghi chú	Ghi chú biến mất khỏi danh sách

⚠ 8. Lưu ý lỗi phổ biến

Vấn đề	Nguyên nhân	Giải pháp
Không hiển thị form PUT/DELETE	HTML form chỉ hỗ trợ GET/POST	Dùng method-override
Form không gửi được dữ liệu	Thiếu name trong input, hoặc thiếu body-parser	Kiểm tra form + dùng express.urlencoded()
Không tìm thấy file EJS	File không nằm đúng thư mục views	Đặt đúng tên, đúng đường dẫn
Không load được CSS	Thiếu express.static('public')	Thêm middleware phục vụ file tĩnh

💡 9. MẸO & TIPS

- Sử dụng layout.ejs để tránh lặp lại header/footer.
- Luôn kiểm tra dữ liệu req.body bằng console.log.
- Dùng try...catch khi thao tác DB để tránh crash.
- Gắn thêm ID vào từng phần tử trong danh sách để xử lý cập nhật/xoá dễ hơn.

- Có thể dùng thêm Flash Message (sử dụng `connect-flash`) để báo lỗi/thành công đẹp hơn.

PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

LUYỆN TẬP

BÀI TẬP 1: TODO LIST CƠ BẢN

Mục tiêu:

- Luyện tập tạo CRUD cơ bản bằng Express và EJS.
- Quản lý danh sách việc cần làm.

Chức năng:

- Thêm một việc cần làm.
- Xoá một việc.
- Hiện thị danh sách todo.

Công nghệ sử dụng:

- Node.js, Express, EJS (không cần MongoDB)
- Dữ liệu lưu trong mảng (RAM)

Cấu trúc thư mục:

pgsql

```
todo-app/  
├── app.js  
├── views/  
│   ├── index.ejs  
│   └── partials/  
│       └── header.ejs
```

Tóm tắt các bước:

1. Tạo server Express, cấu hình EJS.
2. Tạo biến `let todos = []` lưu dữ liệu.

3. Giao diện `index.ejs` hiển thị danh sách + form nhập.
4. Xử lý POST thêm todo.
5. Xử lý GET xóa theo index.

✓ Test case:

TC#	Tình huống	Kết quả mong muốn
1	Thêm một công việc	Công việc hiển thị trên trang
2	Xoá công việc ở dòng đầu	Công việc biến mất
3	Trang load ban đầu chưa có công việc	Hiển thị "Không có công việc"

BÀI TẬP 2: TRANG HỒ SƠ NGƯỜI DÙNG

Mục tiêu:

- Làm quen với truyền dữ liệu qua `res.render`.
- Hiển thị nhiều thông tin động bằng EJS.

Chức năng:

- Hiển thị thông tin một người dùng (name, age, email, hobbies).
- Dùng partial để chia layout.

Công nghệ sử dụng:

- Node.js, Express, EJS

Cấu trúc thư mục:

CSS

```

user-profile/
├── app.js
├── views/
│   ├── profile.ejs
│   └── partials/
│       ├── header.ejs
│       └── footer.ejs

```

Tóm tắt các bước:

1. Tạo server Express.
2. Tạo biến object `user = { name, age, email, hobbies }`.
3. Tạo view `profile.ejs` nhận dữ liệu từ server.
4. Hiển thị dữ liệu bằng EJS.
5. Dùng partial cho header/footer.

✓ Test case:

TC#	Tình huống	Kết quả mong muốn
1	Truy cập <code>/profile</code>	Hiển thị đầy đủ thông tin user
2	Thay đổi <code>user.name</code> trong code	Tên mới được hiển thị
3	<code>user.hobbies</code> là mảng rỗng	Không hiển thị danh sách sở thích

BÀI TẬP 3: DANH SÁCH SẢN PHẨM CƠ BẢN (dùng MongoDB)

Mục tiêu:

- Làm CRUD với MongoDB + EJS.
- Thực hành cấu trúc route, controller, model.

Chức năng:

- Hiển thị danh sách sản phẩm.
- Thêm sản phẩm mới.
- Xóa sản phẩm.

Công nghệ sử dụng:

- Node.js, Express, EJS
- MongoDB + Mongoose

Cấu trúc thư mục:

pgsql

```
product-app/  
├── app.js  
├── models/  
│   └── Product.js  
├── routes/  
│   └── products.js  
└── views/  
    └── products/  
        ├── index.ejs  
        └── new.ejs
```

Tóm tắt các bước:

1. Cài Mongoose, kết nối MongoDB.
2. Tạo model `Product`.
3. Tạo route `/products`, `/products/new`, `/products/:id/delete`.
4. Tạo form thêm sản phẩm.
5. Hiển thị danh sách sản phẩm.

Test case:

TC#	Tình huống	Kết quả mong muốn
1	Gửi form sản phẩm mới	Sản phẩm hiển thị trên danh sách
2	Xóa sản phẩm	Không còn trong danh sách
3	Truy cập <code>/products</code> khi không có dữ liệu	Hiển thị thông báo "Chưa có sản phẩm"